

Día 04 · Tema 2 — Ejercicios: convolución, pooling y transfer learning

Qué es esta hoja. El ejercicio del día de la sesión: uno por sesión, y es el único obligatorio. Es el que entra en el examen con el cálculo incluido, y repite el mismo cálculo del ejemplo resuelto en clase con otros números. Basta con hacer ese para seguir el curso con solvencia.

Dónde está lo demás. Los ejercicios de ampliación de esta sesión van en una hoja aparte, *Ampliación*. Son opcionales, no se piden calculados en el examen y de ellos sólo puede preguntarse la idea, a nivel conceptual.

EL EJERCICIO DEL DÍA obligatorio · entra en el examen

1. Convolución y pooling a mano: el detector de huecos de estante

Una cadena de supermercados instala en sus tiendas una app que fotografía los estantes y avisa cuando hay un *hueco*, es decir, un espacio vacío donde debería haber producto. El primer filtro de la CNN que va dentro de la app busca *bordes verticales*, porque la frontera entre el bloque de producto y el hueco vacío es exactamente una línea vertical.

Trabajamos sobre un recorte de 4×4 píxeles de una de esas fotos, en escala de grises, donde 0 es negro y 9 es blanco:

$$X = \begin{pmatrix} 9 & 8 & 1 & 1 \\ 8 & 9 & 1 & 2 \\ 6 & 5 & 6 & 5 \\ 5 & 4 & 6 & 6 \end{pmatrix}.$$

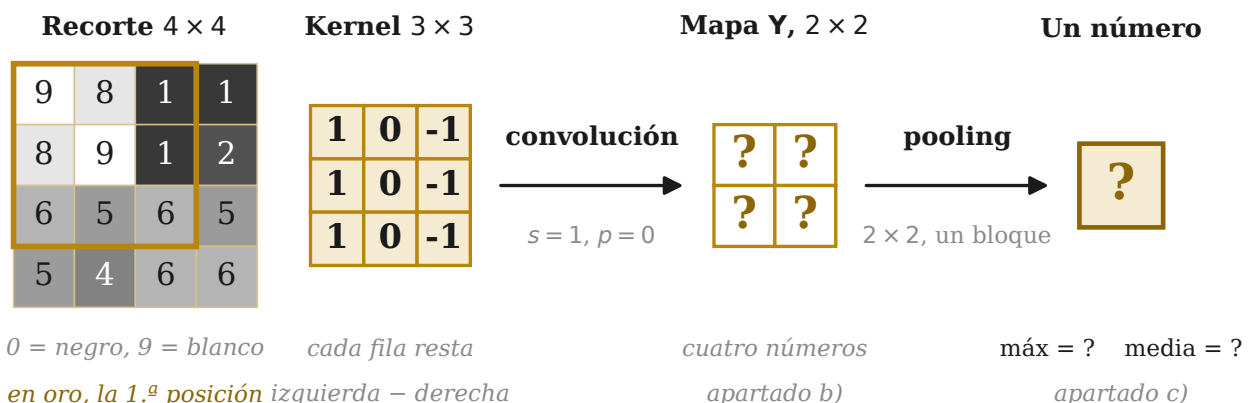
En la mitad de arriba las dos columnas de la izquierda son claras (producto iluminado) y las dos de la derecha oscuras (el hueco vacío, que no tiene producto que refleje la luz). La mitad de abajo es gris intermedio en toda su anchura: ahí hay producto de lado a lado y no hay hueco.

El kernel del filtro es el detector de bordes verticales de las slides:

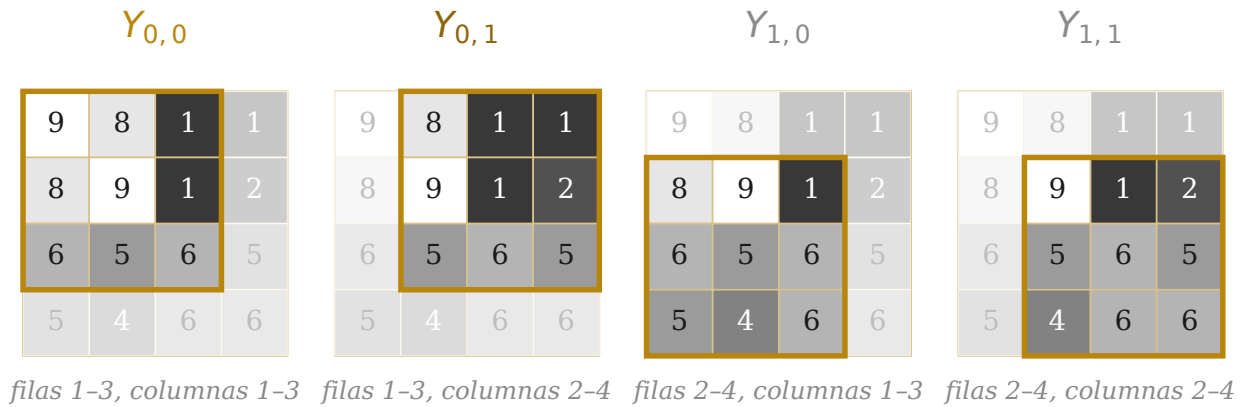
$$K = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}.$$

Leído en palabras: en cada una de sus tres filas resta el píxel de la derecha al de la izquierda e ignora el del centro. Devuelve un número grande y positivo cuando el parche es claro a la izquierda y oscuro a la derecha, y cero cuando el parche es plano.

La capa convolucional va sin *padding* ($p = 0$), con *stride* $s = 1$, sin sesgo y sin función de activación. Detrás de ella va una capa de *pooling* de ventana 2×2 y *stride* 2.



Lo que hay que rellenar, de un vistazo. Los píxeles del recorte están pintados con su nivel de gris real: el bloque oscuro de la mitad superior derecha es el hueco del estante. El kernel se coloca sobre un parche 3×3 de la imagen (en oro, la primera de las cuatro posiciones), y cada posición produce una casilla del mapa Y. La capa de pooling se come el mapa entero y deja un solo número.



Qué parche le toca a cada casilla del mapa. Las cuatro posiciones del kernel, en el orden del apartado (b). En cada una, los píxeles velados son los que *no* entran en esa cuenta: para $Y_{0,0}$ sobra la última fila y la última columna, para $Y_{1,1}$ sobran la primera fila y la primera columna, y así. Cada recuadro dorado son los nueve números que se multiplican por el kernel.

a) **Paso 1: tamaño del mapa de salida.** Con la fórmula

$$H' = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1$$

(y la análoga para W'), di cuántas filas y cuántas columnas tiene el mapa de activación Y que sale de la convolución, es decir, en cuántas posiciones distintas cabe el kernel sobre la imagen. Aquí $H = W = 4$, $k = 3$, $p = 0$ y $s = 1$. Todavía no hace falta calcular ningún valor.

b) **Paso 2: los valores del mapa.** Calcula las cuatro casillas de Y con

$$Y_{i,j} = \sum_{u=0}^2 \sum_{v=0}^2 X_{i+u, j+v} K_{u,v}, \quad i, j \in \{0, 1\},$$

que en la práctica significa colocar el kernel sobre un parche 3×3 de la imagen, multiplicar casilla a casilla y sumar los nueve productos. Los cuatro parches son, en orden: $Y_{0,0}$ usa las tres primeras filas y las tres primeras columnas de X ; $Y_{0,1}$, las tres primeras filas y las tres últimas columnas; $Y_{1,0}$, las tres últimas filas y las tres primeras columnas; y $Y_{1,1}$, las tres últimas filas y las tres últimas columnas. Escribe el resultado como una matriz 2×2 .

c) **Paso 3: pooling.** La ventana 2×2 de la capa de pooling cubre el mapa Y entero en un solo bloque, de modo que esta capa devuelve un único número, que es el que llega al clasificador final.

- i) ¿Qué número sale con *max-pooling* (el máximo del bloque)?
- ii) ¿ Y con *average-pooling* (la media del bloque)?
- iii) ¿Cuántos parámetros entrenables añade a la red esta capa de pooling?

d) **Lectura de negocio.** Ese único número es toda la evidencia de «hay un borde vertical en esta foto» con la que se queda la red. Primero: de los dos resúmenes del apartado (c), ¿cuál mandarías a producción para disparar la alarma de hueco, y qué está diluyendo el otro? Justifícalo señalando qué casillas de Y vienen de la mitad de arriba de la foto y cuáles de la mitad de abajo. Segundo: si el mismo estante se fotografía un poco más abajo, de forma que la frontera producto-hueco cae en otras filas del recorte, ¿cambia mucho el número que sale del max-pooling? ¿Qué propiedad le da eso a la app que un MLP sobre los píxeles en crudo no tendría?